



Algoritmul lui Dijkstra

Drumuri minime în grafuri ponderate

Andrei Mihai Ciobanu

Algolymp Summer School

De la BFS la Dijkstra

Algoritmul lui Dijkstra

Implementare

Corectitudine și limitări

Variații

Recapitulare

De la BFS la Dijkstra

BFS și grafuri ponderate

BFS garantează drumuri minime când toate muchiile au costul 1.

Invariant BFS

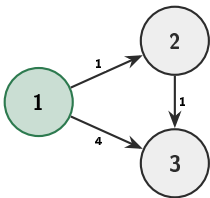
Nodurile se procesează în ordine crescătoare a **numărului de muchii** față de sursă. La runda k : toate nodurile la distanță exact k muchii.

Problema cu costuri diferite

O muchie de cost 5 e tratată identic cu una de cost 1. BFS nu consideră costurile — ordinea de procesare nu mai reflectă distanța reală.

Generalizare: dacă, în loc de numărul de muchii, am procesa nodurile în ordinea **distanței curente față de sursă**, algoritmul ar fi corect pe grafuri ponderate. Aceasta este ideea algoritmului lui **Dijkstra**.

BFS pe grafuri ponderate



3 noduri, 3 muchii

1 → 2: cost 1

1 → 3: cost 4

2 → 3: cost 1

BFS din nodul 1

Runda 1: marchează 2 și 3 (vecini direcți ai lui 1).

BFS returnează: 1 muchie parcursă la nodul 3

Distanță minimă reală: 2

Drumul 1 → 2 → 3 are cost $1 + 1 = 2$.

Drumul direct 1 → 3 are cost 4.

**BFS nu consideră costurile —
returnează număr de muchii, nu cost.**

Dijkstra alege 1 → 2 → 3 cu cost 2. ✓

Algoritmul lui Dijkstra

Dijkstra — ideea algoritmului

Input: graf $G = (V, E)$, costuri $w \geq 0$, sursă s .

Output: $d[u]$ — distanța minimă de la s la u .

Structuri de date:

- $d[u]$ — distanța curentă de la s la u
- $vis[u]$ — dacă nodul u a fost **marcat** (vizitat)

Inițial: $d[s] = 0$, $d[u] = \infty$ pentru $u \neq s$; $vis[u] = \text{false}$ pentru oricare u .

Algoritmul execută n **iterații**. La fiecare iterație:

1. Alege nodul nemarcat u cu $d[u]$ **minim**. Dacă $d[u] = \infty$, nodurile rămase sunt inaccesibile — ne oprim.
2. Marchează u : $vis[u] \leftarrow \text{true}$.
3. **Relaxare**: pentru fiecare muchie (u, v) de cost w : dacă $d[u] + w < d[v]$, actualizăm $d[v] \leftarrow d[u] + w$ (adică vizităm vecinii și le actualizăm distanța dacă putem).

La final, $d[u]$ conține distanța minimă de la s la u pentru orice nod u .

Câte operații facem per iterație?

- **Căutarea minimului** (pasul 1): parcurgem toate cele n noduri pentru a găsi nodul u nemarcat cu $d[u]$ minim — $O(n)$
- **Relaxare** (pasul 3): fiecare muchie e relaxată cel mult o dată — $O(m)$ total

Total pentru cele n iterații:

$$O(n) \times n \text{ iterații} + O(m) = O(n^2 + m)$$

Când este $O(n^2 + m)$ optim?

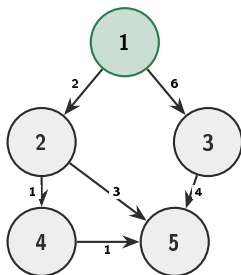
Pe grafuri **dense** ($m \approx n^2$): $O(n^2 + m) = O(n^2)$ — nu putem face mai bine.

Pe grafuri **rare** ($m \ll n^2$): primul termen domină, putem îmbunătăți.

Optimizare pentru grafuri rare

Dacă extragem minimul cu un **min-heap** în loc de scanare liniară: $O(\log n)$ per extragere $\Rightarrow$ total $O((n + m) \log n)$.

Exemplu



Pas	Ales	$d[1]$	$d[2]$	$d[3]$	$d[4]$	$d[5]$
Init	—	0	∞	∞	∞	∞
1	1	0	2	6	∞	∞
2	2	0	2	6	3	5
3	4	0	2	6	3	4
4	5	0	2	6	3	4
5	3	0	2	6	3	4

Rezultat: $dist = [0, 2, 6, 3, 4]$

Drum optim la 5: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$, cost
 $2 + 1 + 1 = 4$.

Implementare

Optimizare pentru grafuri rare

Pe grafuri rare ($m \ll n^2$), termenul $O(n)$ per extragere domină. Soluție: structură de date cu extragerea minimului în $O(\log n)$.

C++ pune la dispoziție două opțiuni:

```
set<pair<int,int>>
```

Suportă ștergerea și extragerea minimului în $O(\log n)$. Stocăm perechi ($\text{dist}[v], v$). La relaxare: ștergem perechea veche, inserăm perechea nouă. `set`-ul conține mereu exact nodurile în așteptare (cel mult n elemente).

```
priority_queue
```

Constantă mai mică decât `set`. Extragerea minimului în $O(\log n)$. **Limitare:** nu suportă ștergerea unui element arbitrar.

priority_queue — duplicate și complexitate

Fără posibilitate de ștergere, la relaxare **nu eliminăm** perechea veche — inserăm pur și simplu perechea nouă ($\text{dist}[v], v$).

Consecință: un nod poate apărea de **mai multe ori** în heap.

- La extragerea (d, v) : dacă $d > \text{dist}[v]$ — intrare **veche**, ignorăm (continue).
- Fiecare relaxare produce un push — cel mult $O(m)$ push-uri total.
- Fiecare push produce exact un pop — deci $O(m)$ pop-uri total.
- Heap-ul crește la $O(m)$ elemente $\Rightarrow$ fiecare push/pop: $O(\log m)$.

Constantă — $O(\log m)$ vs. $O(\log n)$

De cele mai multe ori $m > n$, deci $\log m > \log n$, însă cele două sunt asimptotic identice: $m \leq n^2 \Rightarrow \log m \leq 2 \log n$. În practică priority_queue rămâne preferată: constanta mai mică față de set compensează factorul $\log m$ ușor mai mare.

Total: $O(m)$ push-uri + $O(m)$ pop-uri, fiecare $O(\log m)$:

$$O(m \log m) = O(m \log n)$$

Implementare cu priority_queue — $O(m \log n)$

```
1 int n, m;
2 vector<pair<int, int>> g[nmax + 1]; // g[u] = { { v, w }, ... }
3
4 vector<int> dijkstra(int src) {
5     vector<int> dist(n + 1, inf);
6     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
7     dist[src] = 0;
8     pq.push({ 0, src });
9     while (!pq.empty()) {
10         // extragem nodul cel mai apropiat
11         int d = pq.top().first;
12         int u = pq.top().second;
13         pq.pop();
14         // daca e o intrare outdated (am gasit o distanta mai buna intre timp), abandonam
15         if (d > dist[u]) {
16             continue;
17         }
18         // relaxam
19         for (auto& e : g[u]) {
20             int v = e.first, w = e.second;
21             if (dist[u] + w < dist[v]) {
22                 dist[v] = dist[u] + w;
23                 pq.push({ dist[v], v });
24             }
25         }
26     }
27     return dist;
28 }
```

```
1 int main() {
2     cin >> n >> m;
3     for (int i = 0; i < m; i++) {
4         int u, v, w;
5         cin >> u >> v >> w;
6         g[u].push_back({ v, w });
7         // pentru graf neorientat adaugam si muchia inversa:
8         // g[v].push_back({ u, w });
9     }
10
11     vector<int> dist = dijkstra(1);
12     for (int i = 2; i <= n; i++) {
13         cout << (dist[i] == inf ? -1 : dist[i]) << " ";
14     }
15     return 0;
16 }
```

▷ *Atenție la tipul datelor: dacă $n, m \leq 10^5$ și costurile $\leq 10^9$, distanțele pot depăși int . În acest caz folosim `long long` și `inf = 1018`.*

Restaurarea drumului minim

Pe lângă distanțe, putem reconstitui și drumul: păstrăm $p[v]$ — predecesorul lui v pe drumul minim de la sursă.

Modificare în dijkstra — adăugăm $p[]$ și îl actualizăm la relaxare:

```
1 vector<int> dijkstra(int src) {
2     vector<int> dist(n + 1, inf);
3     vector<int> p(n + 1, -1);    // p[v] = predecesorul lui v
4     // ...
5     for (auto& e : g[u]) {
6         int v = e.first, w = e.second;
7         if (dist[u] + w < dist[v]) {
8             dist[v] = dist[u] + w;
9             p[v] = u;            // actualizam predecesorul
10            pq.push({ dist[v], v });
11        }
12    }
13    // ...
14 }
```

Reconstituire — pornim din t , urmăm $p[]$ până la sursă, inversăm:

```
1 vector<int> get_path(int t) {
2     vector<int> path;
3     for (int v = t; v != -1; v = p[v]) {
4         path.push_back(v);
5     }
6     reverse(path.begin(), path.end());
7     return path; // P = (s, ..., p[p[t]], p[t], t)
8 }
```

Corectitudine și limitări

De ce funcționează Dijkstra?

Afirmație

Când extragem $(dist[v], v)$ din heap, $dist[v]$ este distanța minimă reală de la s la v și nodul v nu va mai fi considerat de acum încolo.

Presupunem că există un drum mai bun $P : s \rightarrow \dots \rightarrow v$ cu $\text{cost} < dist[v]$.

Cazul 1 — P trece doar prin noduri marcate.

Toate relaxările de pe P au fost deja efectuate $\Rightarrow dist[v] \leq \text{cost}(P)$.

Contradicție.

Cazul 2 — P trece printr-un nod nemarcat u .

Deoarece extragem v înainte de u din heap: $dist[v] \leq dist[u]$.

Predecesorul lui u pe P e marcat și deja procesat, deci

$dist[u] \leq \text{cost}(s \rightarrow \dots \rightarrow u \text{ în } P)$.

Costul total al lui P :

$$\text{cost}(P) = \underbrace{\text{cost}(s \rightarrow \dots \rightarrow u)}_{\geq dist[u]} + \underbrace{\text{cost}(u \rightarrow \dots \rightarrow v)}_{\geq 0} \geq dist[u] \geq dist[v]$$

Contradicție — P nu e mai scurt. Deci $dist[v]$ este corect. ✓

De ce nu merge cu costuri negative?

Ipoteza nu mai e validă

Demonstrația s-a bazat pe $\text{cost}(u \rightarrow \dots \rightarrow v) \geq 0$. Cu costuri negative, $\text{cost}(P)$ poate fi $< \text{dist}[v]$ chiar dacă $\text{dist}[v] \leq \text{dist}[u]$ — dar nodul v e deja finalizat și nu mai poate fi revizitat.

Soluție: Bellman-Ford

Relaxează **toate** cele m muchii de $n - 1$ ori. Nu face presupuneri despre ordinea optimă — fiecare iterație poate îmbunătăți orice nod.

- Complexitate: $O(nm)$ față de $O(m \log n)$ la Dijkstra
- Funcționează cu costuri negative
- Detectează cicluri negative (dacă după $n - 1$ iterații mai există relaxări posibile, există un ciclu negativ)

Variații

Multi-source Dijkstra

Problemă: mai multe surse

Vrem distanța minimă de la **oricare** nod din mulțimea S la fiecare nod din graf.

Exemplu: distanța minimă de la orice capitală la fiecare oraș.

Soluție: inițializare multiplă

Inițializăm $dist[s_i] = 0$ pentru toți $s_i \in S$ și inserăm toate sursele în heap de la început. Restul algoritmului e identic.

```
1 vector<int> multi_dijkstra(vector<int>& sources) {
2     vector<int> dist(n + 1, inf);
3     priority_queue<pair<int, int>, vector<pair<int, int>>,
4         greater<pair<int, int>>> pq;
5     for (int s : sources) {
6         dist[s] = 0;
7         pq.push({0, s});
8     }
9     while (!pq.empty()) { /* ... restul identic cu dijkstra */ }
10    return dist;
11 }
```

▷ Echivalent cu adăugarea unui nod virtual 0 cu muchii de cost 0 la fiecare sursă.

Tabel comparativ

Algoritm	Costuri	Complexitate	Observații
BFS	1	$O(n + m)$	—
Dijkstra	≥ 0	$O(m \log n)$	priority_queue
Bellman-Ford	orice	$O(nm)$	detectează cicluri < 0

Când să folosești Bellman-Ford în loc de Dijkstra?

- Costuri negative pe muchii (fără cicluri negative)
- Vrei să detectezi cicluri de cost negativ

▷ Dijkstra cu *priority_queue* pe grafuri rare e $O(m \log n)$; pe grafuri dense alternativa cu extragerea naivă a minimului e mai rapidă.

Recapitulare

Recapitulare

- **Dijkstra** — **S**ingle **S**ource **S**hortest **P**aths (SSSP) pe grafuri cu $w \geq 0$; greedy: extrage la fiecare pas nodul nefinalizat cu distanța minimă
- Cu `priority_queue`: permitem intrări duplicate dar le ignorăm când le extragem din heap
- Costuri negative $\Rightarrow$ **Bellman-Ford**: relaxează toate muchiile de $n - 1$ ori, detectează cicluri negative

Algoritm	Condiție	Complexitate
BFS	$w = 1$	$O(n + m)$
Dijkstra naiv	$w \geq 0$	$O(n^2 + m)$
Dijkstra + pq	$w \geq 0$	$O(m \log n)$
Bellman-Ford	—	$O(nm)$

1. **Dijkstra** (infoarena — Arhiva Educațională)
2. **Drumuri minime** (infoarena — preONI 2006 R4)
3. **Patrol** (infoarena)
4. **Catun** (infoarena)
5. **dragoni** (AlgoCode #52)

1. **CP-Algorithms** — Dijkstra Algorithm — teorie, dovadă, implementare (EN)
2. **CP-Algorithms** — Dijkstra on sparse graphs — varianta cu `priority_queue` (EN)
3. **Infobits Academy F1** — p. 167 (RO)